



CLASE 3 • 3 HS

Decisiones y Compuertas

Que el programa **lea** los pulsadores y **decida** qué hacer. Al salir, el entrenador funciona como seis compuertas lógicas a la vez.



HOY

Seis bloques: el ESP32 escucha y decide

1 • 25 min

Entradas y pull-up

2 • 20 min

bool y comparaciones

3 • 30 min

if, else, else if

4 • 35 min

**Operadores lógicos =
compuertas**

5 • 15 min

De Morgan

6 • 55 min

Ejercicios y TP

Hasta ahora el ESP32 hablaba pero no escuchaba. Hoy **lee entradas** y **toma decisiones**: el álgebra de Boole ya está adentro de C.



Leer un pin: digitalRead

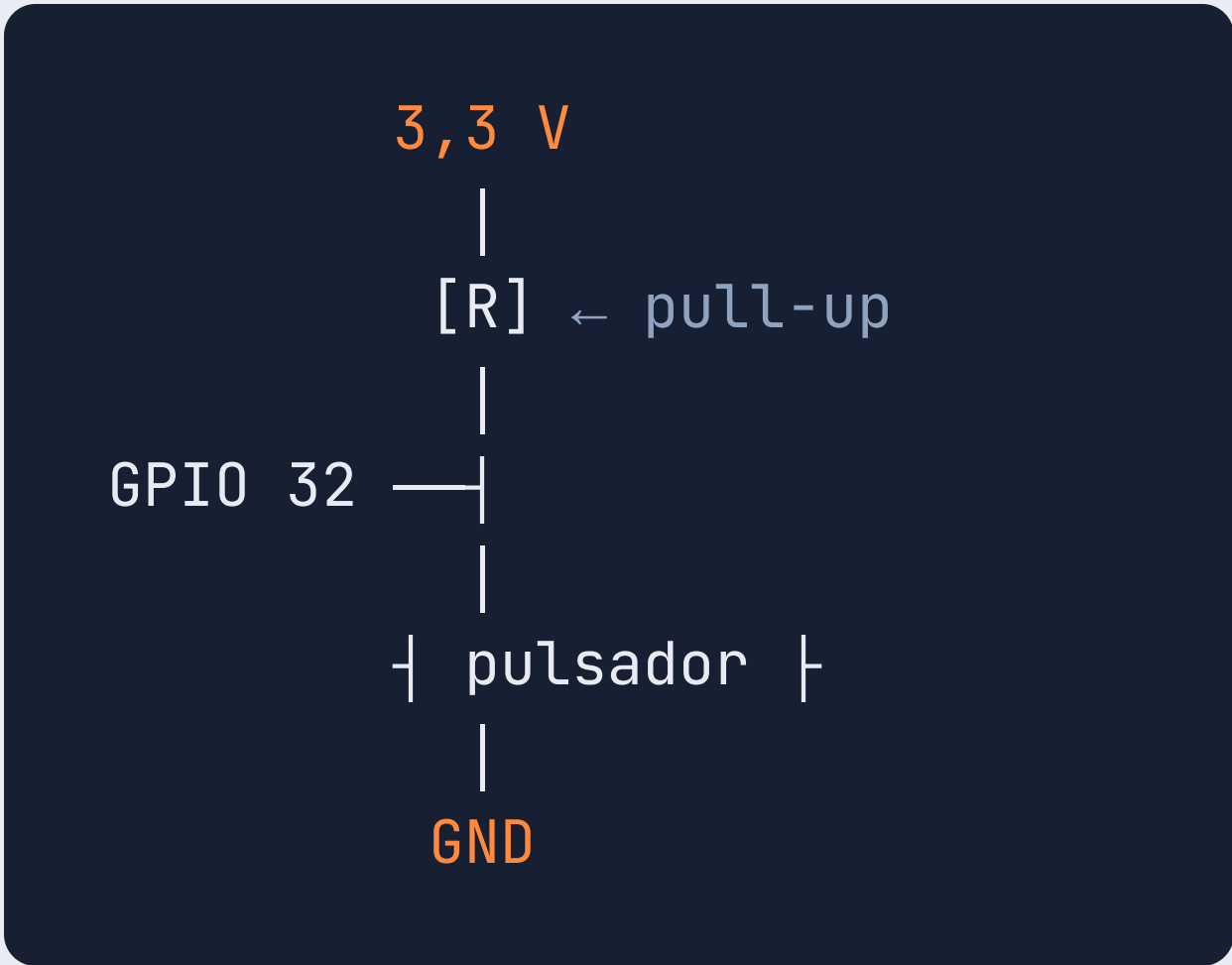
Instrucción	Qué hace
<code>pinMode(32, INPUT_PULLUP);</code>	Configura el pin 32 como entrada con resistencia de pull-up interna. Va en <code>setup()</code> .
<code>digitalRead(32)</code>	Lee el pin: da HIGH (1) si hay tensión, LOW (0) si está a 0 V.

```
int lectura = digitalRead(32);  
Serial.printf("El pin lee %d\n", lectura);
```

`digitalRead` **devuelve un valor**: se puede guardar en una variable, imprimirlo o usarlo en una cuenta.



¿Por qué apretado da 0?



Pulsador	El pin queda en...	digitalRead da
Suelto	3,3 V (a través de la resistencia)	HIGH = 1
Apretado	GND (directo)	LOW = 0

Apretado = 0, suelto = 1.

La resistencia de pull-up está **adentro del ESP32**. Sin ella, con el botón suelto el pin quedaría **flotando** y leería cualquier cosa.



Invertir la lectura una sola vez

```
void loop() {  
  int lectura = digitalRead(BOTON_A);  
  bool a = !lectura; // apretado → 1  
  Serial.printf("Pin: %d    a: %d\n", lectura, a);  
  if (a) {  
    digitalWrite(L0, HIGH);  
  } else {  
    digitalWrite(L0, LOW);  
  }  
  delay(200);  
}
```

Lógica negativa

La señal física vale 0 cuando la condición es verdadera: es **activa en bajo**. El ! de C es la compuerta NOT que la corrige.

Lo mismo, de otra forma:

`digitalRead(BOTON_A) == LOW`



Un bool es un bit

Valor	Significa	Número
true	Verdadero	1
false	Falso	0

```
bool encendido = true;
Serial.printf("%d\n", encendido);
// imprime 1
digitalWrite(L0, encendido);
// true es 1 = HIGH: el LED enciende
```



En C, cualquier número distinto de 0 cuenta como verdadero, y el 0 cuenta como falso.



Comparar da un bool

Operador	Pregunta	Ejemplo con n = 7	Resultado
=	¿Es igual?	n = 7	true (1)
≠	¿Es distinto?	n ≠ 7	false (0)
<	¿Es menor?	n < 10	true (1)
>	¿Es mayor?	n > 10	false (0)
≤	¿Es menor o igual?	n ≤ 7	true (1)
≥	¿Es mayor o igual?	n ≥ 8	false (0)

Una comparación es una **pregunta** que el programa responde con **1 o 0**.



= y == son cosas distintas

`n = 7`

Asignación: guardar 7 en n

`n == 7`

Comparación: ¿n vale 7?

```
int n = 3;
if (n = 7) {
    Serial.printf("n vale 7\n");
}
Serial.printf("n = %d\n", n);
```

¿Qué imprime?

Guarda 7 en n, y como 7 es distinto de 0, la condición es **siempre verdadera**. El compilador no da error: a lo sumo un *warning*.



if / else: una cosa u otra

```
if (a) {  
    digitalWrite(L0, HIGH);  
} else {  
    digitalWrite(L0, LOW);  
}
```



Siempre se ejecuta **uno solo** de los dos bloques, nunca los dos.



else if: más de dos caminos

```
if (n < 5) {  
    Serial.printf("y chico\n");  
} else if (n < 15) { // n ≥ 5  
    Serial.printf("y mediano\n");  
} else {           // n ≥ 15  
    Serial.printf("y grande\n");  
}
```

Para pensar

Con **n = 3**, ¿por qué no imprime "mediano", si 3 también es menor que 15?

Las condiciones se prueban **en orden, de arriba hacia abajo**. Apenas una es verdadera, se ejecuta su bloque y se saltan todas las demás. En el ejemplo, **n % 2 == 0** pregunta "¿es par?".



Las reglas de escritura del if

Regla	Bien	Mal
La condición va entre paréntesis	<code>if (n > 3)</code>	<code>if n > 3</code>
No lleva ; después del paréntesis	<code>if (n > 3) {</code>	<code>if (n > 3); {</code>
Usar siempre llaves, aunque sea una sola línea	<code>if (a) { x++; }</code>	<code>if (a) x++; y++;</code>
Comparar es ==	<code>if (n == 0)</code>	<code>if (n = 0)</code>



`if (n > 3);` significa "si $n > 3$, no hacer nada". Las llaves que vienen después se ejecutan **siempre**, y el compilador no se queja.



Con &&, || y ! se arman todas

Compuerta	Álgebra	En C	AB = 00	01	10	11
AND	$A \cdot B$	<code>a && b</code>	0	0	0	1
OR	$A + B$	<code>a b</code>	0	1	1	1
NOT	$\overline{A}$	<code>!a</code>	1	1	0	0
NAND	$(A \cdot B) \overline{}$	<code>!(a && b)</code>	1	1	1	0
NOR	$(A + B) \overline{}$	<code>!(a b)</code>	1	0	0	0
XOR	$A \oplus B$	<code>a != b</code>	0	1	1	0
XNOR	$(A \oplus B) \overline{}$	<code>a == b</code>	1	0	0	1

No hay operador para XOR: con bool, **XOR** es "son distintas" (`a != b`) y **XNOR** es "son iguales" (`a == b`).



El ESP32 como compuertas lógicas

```
void loop() {  
  bool a = digitalRead(BOTON_A) == LOW;  
  bool b = digitalRead(BOTON_B) == LOW;  
  digitalWrite(L0, a && b);      // AND  
  digitalWrite(L1, a || b);      // OR  
  digitalWrite(L2, !a);          // NOT  
  digitalWrite(L3, !(a && b));    // NAND  
  delay(300);  
}
```

Antes de apretar nada

Con A y B sueltos, ¿qué LEDs van a estar encendidos? Después recorrer la tabla apretando A, B y los dos.

Un bool vale 0 o 1: se pasa **directo a digitalWrite**. No hace falta un if/else, aunque haría lo mismo.



De la teoría a C, símbolo por símbolo

• → &&

+ → ||

barra → !

$F = A \cdot B + \bar{C} \rightarrow f = (a \ \&\& \ b) \ || \ !c;$

Como en el álgebra, el AND tiene prioridad sobre el OR, pero conviene **usar paréntesis** igual. Y no confundir && con &, ni || con |: son otros operadores (Clase 5).



Leyes de De Morgan, en vivo

Ley	Álgebra	En C
1	$(A \cdot B)^{\overline{}} = \overline{A} + \overline{B}$	<code>!(a && b) == (!a !b)</code>
2	$(A + B)^{\overline{}} = \overline{A} \cdot \overline{B}$	<code>!(a b) == (!a && !b)</code>

```
bool izquierda = !(a && b);  
bool derecha   = !a || !b;  
Serial.printf("izq=%d der=%d\n",  
              izquierda, derecha);
```

Para pensar

Si por error se escribe `!a && !b` del lado derecho de la ley 1, ¿en qué combinación se descubre?



Ahora ustedes

EJERCICIO 1

Funciones de 3 entradas

Con A, B y C. En L0, la **mayoría** (dos o más en 1): tabla de 8 filas y suma de productos. En L1, **$F = A \cdot B + \bar{C}$** . Imprimir cada medio segundo.

EJERCICIO 2

Alarma

B = armada, A = puerta abierta, C = pánico. Pánico, o armada y puerta: L0–L3 parpadean y *** ALARMA ***. Si no, armada: L4. Si no: Desarmada.

EJERCICIO 3

Comprobar De Morgan

Los dos lados de las dos leyes en L0–L3 (un par de LEDs por ley). Con un if, imprimir OK o NO COINCIDEN. Recorrer las 4 combinaciones.

Apretar los botones para cada fila es tedioso. Está bien: en la Clase 4 el programa recorre la tabla solo.



Mi entrenador es un laboratorio de compuertas

1. Partir del TP de la Clase 2 (cartel y pines), con el encabezado actualizado.
2. Leer A y B con INPUT_PULLUP y pasarlos a bool: apretado = 1.
3. Mostrar cada compuerta en su LED.
4. Imprimir el estado cada medio segundo, en una línea.
5. Verificar con los botones las 4 filas de cada tabla de verdad.

L0 AND	L4 NOR
L1 OR	L5 XNOR
L2 XOR	L6 A
L3 NAND	L7 B

A=1 B=0 | AND=0 OR=1 XOR=1 NAND=1 NOR=0 XNOR=0

Entrega: carpeta **SL2026-Apellido-Clase3** en ZIP, sin la carpeta .pio.



Al terminar la clase, sé...

- ✓ leer un pulsador con digitalRead y explicar por qué apretado da 0
- ✓ decir qué valores tiene un bool y cuánto valen como número
- ✓ explicar la diferencia entre = y ==
- ✓ escribir un if / else if / else sin ; de más y con sus llaves
- ✓ escribir en C cualquier compuerta de dos entradas
- ✓ traducir una expresión de la teoría (como $A \cdot B + \bar{C}$) a C
- ✓ comprobar una ley de De Morgan con el ESP32

La semana que viene: hoy recorrieron las tablas apretando botones. En la Clase 4 el ESP32 las va a **imprimir enteras, solo**, para cualquier función.

